PROJECT SYNOPSIS

Blockchain-Based Digital Evidence Vault

Detailed synopsis of the Proofcase Next.js MVP

1. Abstract

The Blockchain-Based Digital Evidence Vault is a web application project for organizing digital evidence and documenting its integrity and handling history. The application provides a dashboard where evidence records can be searched and reviewed, an upload flow that associates files with an investigation, and a detail view that displays file metadata, a SHA-256 fingerprint, verification status, and custody events. The current MVP calculates hashes in the browser and stores uploaded files and records locally. It links the hashes of custody events to form a local sequence that can be checked for consistency. The project demonstrates the workflow and core concepts of a digital evidence vault; the current implementation does not connect to a blockchain network. A production system would require secure backend storage, access controls, durable audit services, and an explicitly selected blockchain integration.

2. Introduction and Background

Digital evidence may include documents, images, video, email, system logs, and network captures. An investigation can accumulate many files, and those files may pass between investigators, analysts, and legal staff. Teams need to know which case a file belongs to, who added it, when it entered the evidence collection, and whether its contents have changed since intake.

A cryptographic hash provides a compact fingerprint of file contents. If the file changes, its calculated fingerprint will normally change as well. A chain-of-custody record complements that fingerprint by documenting events associated with the evidence. Blockchain technology is one possible way to publish or distribute commitments to such records, but a user interface and a hash-linked event list alone do not provide a public, decentralized, or immutable ledger. This distinction is reflected in the current prototype.

3. Problem Statement

Evidence stored in ordinary folders or tracked through informal notes can become difficult to search, verify, and explain. Files may be separated from their case context, accidental changes may go unnoticed, and handling events may not be recorded in a consistent format. The project addresses this workflow problem by providing structured evidence records, automatic fingerprints, and a visible custody history. The MVP is intended to demonstrate these functions and provide a basis for future system development.

4. Aim and Objectives

The aim is to develop a simple web-based vault that demonstrates digital evidence intake, integrity checking, and chain-of-custody review.

- Provide a dashboard for viewing and searching evidence records.
- Associate each evidence record with a case and an evidence category.
- Generate a SHA-256 fingerprint from the contents of a newly uploaded file.
- Create a chronological custody record for important evidence-handling actions.
- Link custody-event hashes to the previous event and the evidence fingerprint.
- Allow the user to check stored evidence and download the original file.
- Identify the gap between a local prototype and a production blockchain-based solution.

5. Scope of the Project

The current scope is a single-browser MVP. It supports a sample case dashboard, evidence search and status filtering, file upload, case and category fields, metadata display, fingerprint copying, integrity checking, custody-event review, and download of files saved in the same browser. The interface contains sample evidence records for demonstration. Those sample records have no corresponding local file blobs, so they cannot be verified or downloaded as original files.

The current scope does not include user accounts, multiple investigators sharing a case, server-side evidence storage, external database persistence, legal signatures, a blockchain wallet, a smart contract, or public-chain transaction submission. Those features are future development possibilities, not implemented functions.

6. Proposed System

The proposed system gives an investigator a case-oriented workspace for registering evidence and reviewing its current integrity information. The interface is organized around an evidence list and an evidence detail panel. A user can search by name, record identifier, category, owner, or case, and can narrow the displayed list by verification status. Selecting a row opens the record details and custody history.

Evidence intake

The user selects a file, enters a case identifier, and chooses an evidence category. The browser reads the file bytes and calculates a SHA-256 digest. The file blob is saved in IndexedDB, and the record containing its identifier, filename, file type, category, size, hash, date, owner, case, and status is saved in LocalStorage.

Custody recording

For a new upload, the current application creates intake, hash-verification, and sealing events. Each event includes an action, actor, timestamp, description, previous event hash, and event hash. The event hash is calculated using the previous event hash, event information, timestamp, and evidence fingerprint. The first event uses a genesis value as its previous hash. This creates a linked record within the browser for demonstration and consistency checking.

Integrity verification

When verification is requested, the application checks the stored custody-event links for the selected record. If the original uploaded file is available locally, it recalculates the file digest and compares it with the recorded SHA-256 value. A mismatch changes the record to a pending state and displays a notice. If a sample record has no local file, the interface explains that the original is unavailable rather than claiming that verification succeeded.

7. User Roles and Use Cases

The MVP models a primary user as an investigator working with one case. It does not implement authentication or enforce role-based permissions. The following use cases define the intended interaction:

- Review a case: open the dashboard and see evidence counts, verification status, and recent activity.
- Find a record: search for a filename, case identifier, category, owner, or evidence identifier.
- Add evidence: upload a local file, enter case metadata, and create a fingerprinted evidence record.
- Inspect evidence: view its metadata, fingerprint, custody events, and event hashes.
- Verify evidence: check that the custody links and locally stored file fingerprint match the record.
- Retrieve evidence: download the original file from the same browser where it was stored.

8. Functional Requirements

- FR-1: The system shall display evidence records with filename, type, size, case, uploader, date, and verification status.
- FR-2: The system shall allow the user to search and filter visible evidence records.

- FR-3: The system shall accept a file and evidence metadata through an upload form.
- FR-4: The system shall calculate a SHA-256 fingerprint for a newly uploaded file.
- FR-5: The system shall persist new file contents in browser IndexedDB and record metadata in LocalStorage.
- FR-6: The system shall create and display custody events for newly added evidence.
- FR-7: The system shall validate event links and compare the available file with its stored fingerprint.
- FR-8: The system shall provide actions to copy the fingerprint and download the locally stored file.

9. Data Model and Information Flow

An evidence record in the current application contains an evidence ID, filename, file type, category, file size, SHA-256 digest, added date, owner, verification status, case ID, and an ordered list of custody events. Each custody event contains an action, actor, date, description, previous hash, and block hash. The term block hash in the interface refers to a hash-linked event in this local prototype; it does not mean that a blockchain block has been mined or accepted by a network.

Information flow

- File selection: the browser receives a file selected by the user.
- Fingerprint generation: Web Crypto calculates SHA-256 from the file bytes.
- Local persistence: IndexedDB stores the file; LocalStorage stores its record and custody events.
- Record display: the dashboard reads the saved record and renders its status and metadata.
- Verification: the application recalculates event hashes and, when present, the file hash.

This design is appropriate for a front-end demonstration but has limitations. LocalStorage is not a database with access controls, and the browser profile is not a trusted evidence archive. A real deployment should separate file storage, metadata persistence, identity, audit logging, and blockchain anchoring into components with explicit security boundaries.

10. System Architecture and Modules

Presentation module

The Next.js and React interface renders the overview, evidence table, upload modal, and detail drawer. Responsive CSS adapts the layout for smaller screens. Search, filters, modal state, and selected-record details are managed in the browser.

Evidence service in the browser

Client-side functions calculate file and custody hashes, store and retrieve blobs from IndexedDB, persist record metadata in LocalStorage, and compare the stored file with its recorded digest. These operations currently run in the browser rather than through a server API.

Storage layer

IndexedDB holds binary file content and LocalStorage holds JSON evidence records. These stores are scoped to the browser origin and device. They do not provide synchronization across investigators or automatic backups.

Blockchain integration layer - future

A future backend or smart-contract adapter could anchor selected hashes to a blockchain. The application would need to define what is published, who submits transactions, how transaction confirmation is displayed, how sensitive case metadata is kept off-chain, and how corrections or legal retention requests are handled.

11. Technology Stack and Requirements

- Frontend framework: Next.js with React and TypeScript.
- Cryptography: browser Web Crypto API using SHA-256.

- Browser persistence: IndexedDB for file blobs and LocalStorage for evidence records.
- Interface styling: responsive CSS and Lucide icons.
- Development tools: Node.js and npm for dependency installation and local development.

The user needs a modern browser with IndexedDB and Web Crypto support. Development requires Node.js and npm. The repository provides npm scripts for running the development server, building the application, and starting the built application.

12. Security, Privacy, and Integrity Considerations

The project handles evidence-like content, so integrity, confidentiality, and provenance must be considered together. A hash can help detect a change only when the original hash and its record are protected. If an attacker can replace both the file and the locally stored record, the browser-only prototype cannot provide a trusted proof of the original contents.

- The current system stores user files in browser storage and does not encrypt them at rest.
- There is no authentication, authorization, session management, or separation between users.
- Browser data may be cleared, modified, or lost if the browser profile or device is unavailable.
- The local hash-linked custody sequence is not independently witnessed and is not immutable.
- A production system should keep sensitive evidence content and identifying metadata off a public blockchain.
- A future design should define key management, role permissions, retention, backups, audit access, and export integrity.

For a production implementation, evidence hashes could be anchored externally while evidence files remain in controlled encrypted storage. An independent, append-only audit service and transaction receipts would make later verification more credible. Any blockchain deployment should be selected only after privacy, governance, cost, and operational requirements are documented.

13. Non-Functional Requirements

- Usability: common tasks such as searching, uploading, and inspecting evidence should be clear and require few steps.
- Integrity: the interface should distinguish verified files, pending checks, missing originals, and failed checks.
- Reliability: the application should handle unavailable browser storage and malformed saved records without crashing.
- Performance: hashing and file upload feedback should remain understandable for larger files; future versions may need streaming or server-side processing.
- Compatibility: the interface should support current desktop and mobile browser sizes.
- Maintainability: evidence, hashing, persistence, and presentation concerns should be separable as the prototype grows.

14. Development Methodology

The project is being developed as an incremental web application. The initial stage establishes the Next.js project structure and interface. Subsequent implementation adds evidence records, upload handling, browser persistence, cryptographic fingerprints, custody event links, and detail interactions. A production-oriented next stage would replace sample data and local-only storage with a documented backend and security model before integrating any blockchain network.

The development process should keep implemented behavior distinct from proposed extensions. For this MVP, the deliverable is the browser-based interface and local workflow. Blockchain submission, multi-user operation, secure centralized storage, and legal evidentiary guarantees remain outside the implemented scope.

15. Verification and Evaluation Plan

The following checks are proposed to evaluate the MVP and are not claims that a formal test suite has already been completed:

- Add a file, reload the page, and confirm its record, fingerprint, custody events, and stored blob remain available.
- Verify matching file and event hashes, then modify a file or event hash and confirm that the mismatch is detected.
- Search by filename, ID, case, category, and owner; confirm each status filter returns the expected records.
- Download an uploaded file and compare it with the source; confirm sample records report that the original is unavailable.
- Review upload, search, details, and verification at desktop and mobile viewport sizes.

16. Feasibility and Expected Outcome

The MVP is technically feasible as a local demonstration because it uses a mainstream web framework and browser APIs, without requiring an external service or blockchain node. The expected outcome is a working front-end workflow for evidence registration, fingerprint generation, local file retrieval, hash-linked custody events, and integrity review. The dashboard supports familiar investigator tasks, but operational use would require clear procedures for evidence intake, device handling, access, transfer, export, retention, and incident response. Production readiness would also require secure identity, shared access, reliable storage, audit retention, and blockchain operations.

17. Limitations and Risks

The principal limitation is that browser storage is controlled by the client and is not a trustworthy archival system. Data is not automatically synchronized or backed up, there is no access control, and no blockchain transaction is performed. A hash match demonstrates equality with the stored reference digest, but by itself does not prove who collected the file, when it was collected, or whether the collection process was lawful or forensically sound. The sample dashboard data is illustrative and must not be mistaken for real case evidence.

18. Future Enhancements

- Add authenticated users, investigator roles, and case-based access policies.
- Move metadata to a durable database and files to encrypted, access-controlled storage with backups and retention rules.
- Add an append-only audit service, evidence export reports, and secure multi-user case collaboration.
- Select a blockchain network and anchor hashes or Merkle roots with transaction receipts and independent verification tools.
- Review security, privacy, usability, and operating procedures before real evidence is stored.

19. Conclusion

The Blockchain-Based Digital Evidence Vault project provides a structured starting point for managing digital evidence and demonstrating file integrity checks. Its Next.js MVP includes the main user-facing workflow for adding files, attaching case information, reviewing SHA-256 fingerprints, and inspecting linked custody events. The implementation is intentionally described as a browser-local prototype rather than a deployed blockchain system. By documenting its current boundaries and future requirements, the project can be extended toward a secure, multi-user evidence platform with independently verifiable audit records and carefully designed blockchain anchoring.